How to become an Arborist: An introductory guide

Bao Minh Tran

Deakin University
s224236373@deakin.edu.au

Abstract

Tree is a nice class of graphs that has a long history of development in the field of Computer Science. It has contributed vastly to many areas and been known to be an efficient data structure. Therefore, many exciting ideas have emerged, either organizing data by preserving some nice properties of Trees or proposing specific algorithms to streamline computations on tree-like data structures. In this article, several advanced algorithms and techniques are reviewed by leveraging the distinctive properties offered by a tree. These are Euler Tour technique, Lowest Common Ancestor, Heavy-Light Decomposition, and Centroid Decomposition. These algorithms and techniques are of paramount importance when it comes to problem–solving tasks requiring computations of subgraphs of a tree.

1. Introduction

Graph theory is a matter of study about [1] the relationships between pairs of objects. Dated back to 1735 when it was first recognised by Euler, who propsed a solution to the Konigsberg bridges problem [2]. Since there, the field has received a remarkable volume of contributions and witnessed several ground–breaking ideas, e.g. The Four–color theorem (1977) [3], Hall's theorem (1935) [2], Network flow first publication (1956) [2], Single–source Shortest Path Finding Algorithm (1959) [2], and many more. This matter of study has contributed towards the development of multiple disciplines and become a problem–oriented approach for various optimization problems [4] [1]. In the era of AI, this foundational theory has been extended beyond the classical framework and leveraged for Graph Neural Networks [5].

Throughout the history of Graph theory, esteemed researchers have classified graphs into several sub–types by preserving certain nice properties, including but not limited to [2] Trees (1857), Bipartite Graphs (1916), and Planar Graphs [4]. In this article, we will be exploring deeper about some advanced algorithms on Trees, i.e. Euler Tour technique, Lowest Common Ancestor (LCA), Centroid Decomposition, and Heavy–Light Decomposition (HLD) and certain interesting properties of this type of graph. Also, we make our best effort to support these theorems, corrolaries and lemmas by relatively formal proofs.

Briefly, about the structure of this article. Firstly, we recall certain pivotal definitions, theorems and corrolaries revolving around the central topics within this report. In the sections followed afterwards, we introduce you through various topics, i.e. Euler Tour technique, LCA, Centroid Decomposition, and HLD. In each section, we establish the theoretical foundations, their algorithmic design and certain useful properties.

Since this article involves moderately complicated mathematical knowledge, we assume you have basic understandings about some mathematical notations and graph in Computer Science. Still, we recall certain useful graph theory knowledge in the upcoming section. Note that, throughout this article, we utilise the adjacency list to represent the graph data structure [1].

2. Prerequisites

Before proceeding further, let us introduce some notations used throughout this article. A Graph is a discrete data structure, denoted as [6] $G = (V, E)$, where V is the set of vertices and $E \subseteq V \times V$ is the set of edges. Noticeably, throughout this article, vertex and node are used interchangeably. Next, we introduce some useful definitions below.

Definition 2.1 (A path [6])**.** A path connecting two endpoints $p_0, p_k \in V$ is a non–empty subgraph of $G = (V, E)$, denoted as $P = (V', E')$, where:

$$V' \subseteq V \quad \text{and} \quad E' \triangleq \{(p_0, p_1), \cdots, (p_{k-1}, p_k) \mid (p_i, p_{i+1}) \in E\}$$

Definition 2.2 (A cycle and a circuit [7])**.** For $G = (V, E)$, a cycle C is a path P where two endvertices p_0 and p_k are the same, neither repeated edges nor nodes are allowed. G is called a cyclic graph if there exists at least one cycle, otherwise known as an acyclic graph. Meanwhile, a circuit C is a loosing notion, which allows repeated vertices involved in C.

Definition 2.3 (Connectivity [6])**.** A graph $G = (V, E)$ is called connected if and only if, $\forall s, t \in V : \exists P$, defined by **Def. 2.1**, which links s and t.

Definition 2.4 (Directed vs. Undirected [1])**.** A graph $G = (V, E)$ is called directed if for $(u, v) \in V : (u, v)$ is ordered, i.e. u goes to v but v goes to u is not inherently guaranteed. On the contrary, G is deemed undirected if (u, v) is unordered, i.e. $(u, v) \in E \Leftrightarrow (v, u) \in E$.

Definition 2.5 (A tree [6])**.** A tree $T = (V, E)$ is an undirected **(Def. 2.4)**, connected **(Def. 2.3)** and acyclic **(Def. 2.2)** graph.

Corollary 2.1 (Uniqueness of paths)**.** For a tree $T = (V, E)$, of n vertices, there are exactly $n-1$ edges. Therefore, there is a unique path between arbitrary pairs of nodes.

Proof. Given a tree $T = (V, E)$. We introduce a new edge (u, v) such that $u, v \in V$ and not necessarily adjacent. Since T is connected, i.e. there exists a path connecting u and v. By complementing a new edge (u, v), we introduce a cycle which involves u and v, this violates the constraints of a tree defined in **(Def. 2.5)**. Hence, there are exactly $n - 1$ edges. □

Definition 2.6 (Neighbours and Degree of a vertex [6])**.** In a non–empty graph $G = (V, E)$, a neighbour of a vertex $v \in V$, denoted as $N(v)$, is defined by:

$$N(v) \triangleq \{u \mid (v, u) \in E\}$$

The degree of this vertex, denoted as $\deg(v)$, is given by:

$$\deg(v) \doteq |N(u)|$$

where $|\cdot|$ refers to the number of elements in $(\cdot)$.

Since we are working solely with Trees, the degree of a vertex implicitly refers to all edges incident to that vertex. Note that, there are definitions of out–degree and in–degree for directed graphs. In these cases, $\deg(v)$ is the sum of out– and in–degrees.

Corollary 2.2 (Summation of degrees [1])**.** For $G = (V, E)$ with $|V| = n$ and $|E| = m$:

$$\sum_{v \in V} \deg(v) = 2m$$

Proof. Consider an edge (u, v), we know that the degree of a vertex count the number of edges incident with that vertex. Since 2 vertices form an edge, the sum of their degrees must be twice the number of edges. □

Corollary 2.3 (Odd degree and vertices [8])**.** In all graphs, the number of vertices with odd degree is even.

Proof. From Corollary 2.2, we can rewrite the equation as:

$$|E| = m = \frac{1}{2} \sum_{v \in V} \deg(v)$$

Since $|E|$ must be an integer, and hence $\sum_{v \in V} \deg(v)$ must be divisible by 2, or even. This implies the total number of vertices with odd degrees must be even to ensure their summation of degrees is even, and thus, the proof is completed. □

Theorem 2.1 (Cycle Existence [7])**.** *A graph $G = (V, E)$ has a cycle if $\forall v \in V : \deg(v) \geq 2$.*

Proof. Suppose P is the longest path, or so–called maximal path [7], between $s, t \in V$. Consider s and t, which are the endvertices of P. Remarkably, P is the maximal path, and thus, cannot be extended. While $\deg(s) \geq 2$ and $\deg(t) \geq 2$, this implies there exists another vertex which is adjacent to s or t, and hence P can be extended. This violates the assumption that P is the maximal path. Therefore, s and t must induce a cycle. □

Definition 2.7 (A rooted tree [1])**.** Given a tree $T = (V, E)$, it is called a rooted tree at r, if r has no parent nodes $v \in V$.

Definition 2.8 (A rooted (sub)tree [1])**.** For a tree $T = (V, E)$, for a vertex $v \in V$, a subtree rooted at v, denoted as $S_v = (V_v, E_v) \subseteq T$, contains all descendants of v in T, including itself.

Throughout the discourse in this article, we will assume, without loss of generality, a tree $T = (V, E)$ consists of n nodes, defined by $V \triangleq \{1, 2, \cdots, n\}$ and rooted at node 1.

3. Euler Tour technique

First and forthmost, let us formally define what an Euler Tour is. Simply put, this technique flattens our initial tree into a linear array, and thus, extremely efficient for queries about subtrees. Let immerse ourselves into the theoretical foundation of this technique.

3.1. Theoretical foundation

Definition 3.1 (An Euler Tour and An Eulerian Graph [6])**.** For a graph $G = (V, E)$, an Euler Tour is a circuit that traverses every edge in G. Thus, G is called an Eulerian graph if there exists an Euler Tour.

Theorem 3.1 (Eulerian Graph theorem [7])**.** *$G = (V, E)$ is an Eulerian if and only if G is connected and every vertex $v \in V$ has even degree.*

Proof. We divide our proof into two parts. The first part of the theorem, or the necessary condition can be proved as follow. Suppose G is Eulerian, i.e. there exists an Euler Tour. Obviously, an Euler tour is a circuit and requires traversal through all edges, and hence, G is connected. Meanwhile, a circuit requires the starting and ending

vertices must be the same, thus, all vertices must have out-degrees equals to in–degrees, which implies an even degree in total. Otherwise, it is possible to going through all edges by starting at vertex with odd degrees but impossible to get back to it at the end of the traversal.

For the sufficient condition, suppose G is connected and every vertex in G has even degree. As advised in many references [6] [7] [8], we may want to approach the proof using induction methodology. First, our base case is 1 vertex or $|E| = 0$ or $|E|$ is even if G contains self-loops, and thus, trivially an Eulerian graph. Now, suppose that the sufficient condition holds, or G is an Eulerian graph for $|E| \leq k$. Next, we move to the case where G contains $|E| = m > k$ edges. By **Theorem 2.1**, G is guaranteed to have a cycle, namely C. By excluding the edges in C, we obtain a new graph $G' = G \setminus C$, since C is a cycle, every vertex $c \in C$ cannot be visited more than once, and thus, $\deg(c)$ is reduced by 2. Since $C \subseteq G \Rightarrow c \in C : c \in G$, and hence, G' still contains vertices of even degree with fewer edges. Repeatedly doing this until $|G'| \leq k$, the sufficient condition holds. □

Having discussed the definition of Euler Tour, we may realize that a Tree cannot have an Euler tour, unless it is a simple path. Alternatively, a tree can only be an Eulerian graph if there is a path consisting of distinct edges and vertices [1]. Nevertheless, we can generalize **Theorem 3.1** for directed graph. In directed graphs, the degree of a node becomes the sum of incoming and outcoming edges to and from that node, respectively. Therefore, we can convert the undirected edges into directed edges, indeed, this process is straightforward, by convention, undirected edges allow going back and forth, or implicitly involving two opposite directed edges.

Obviously, this conversion only introduces at most one additional degrees to the endvertices. Additionally, tree is connected, and thus, the conversion above induces a new strongly connected graph. Thus, **Theorem 3.1** always holds for a tree [9].

3.2. Algorithmic design

According to the claims above, in this technique, we firstly convert bidirectional edges into unidirectional edges. This, however, is implicitly given by our adjacency list representation. Supposing we are given a rooted tree, i.e. a tree with a predefined root, and the nodes are indexed consecutively, i.e from 1 to n, or 0 to $n - 1$. Note that, this algorithm still works well for any arbitrary node.

About the algorithm, we maintain two auxiliary arrays, called `time-in[]` and `time-out[]`. Simply put, for a vertex $v \in V$, the `time-in[]` retains the moment when we first visit v and the `time-out[]` stores the moment when we complete visiting all nodes located in the sub–tree rooted at v. To update these two arrays, we definitely use a well–known traversal algorithm, i.e. Depth–First Search (DFS). Briefly, an Euler Tour algorithm is an extension of pre- and post–order traversal algorithms [1]. Indeed, there are various implementation of Euler Tour, each of which serves their own sakes. In this article, we present one Euler Tour variant, which show applicability to the later sections.

Algorithm 1 Euler Tour Technique [10] [9] [1]

```
1: function EULERTOUR(root)
2:     timer ← 0
3:     DFS(root, −1)
4: end function
```

```
1: function DFS(u, parent)
2:     time-in[u] ← timer
3:     euler[timer] ← u                ▷ stores the order of vertices visited in the tour
4:     timer ← timer + 1
5:     for all v ∈ adjacency-list[u] do
6:         if v ≠ parent then
7:             DFS(v, u)
8:         end if
9:     end for
10:    time-out[u] ← timer
11: end function
```

Complexity Analysis In this algorithm, both time and space complexity are amount to the implementation of DFS algorithm, which was already proven to be in $\mathcal{O}(|V| + |E|)$ [1].

3.3. Properties

Now, let us claim some interesting results of this technique, noted as lemmas.

Lemma 3.1 (Contiguous subtree interval)**.** *After running the Euler Tour algorithm on a rooted tree T, for vertex x in the subtree rooted at u, denote $t_x \triangleq \{time \mid euler[time] = x\}$, then $t_x \subseteq [\texttt{time-in}[u], \texttt{time-out}[u])$.*

Proof. For x a child of the subtree rooted at u, for $time_0 \in t_x$, such that $time_0 \notin [\texttt{time-in}[u], \texttt{time-out}[u]]$. This means x must be visited either before or after u. Obviously, this contradicts our initial assumption, where x is a child of subtree rooted at u. As $DFS(x)$ is only performed after it visits u, and no longer performed when it completes visiting every node in the subtree rooted at u. Thus, the lemma must hold. □

Lemma 3.2 (Ancestor query [10] [9])**.** *For $u, v \in V$: u is an ancestor of v if and only if $\texttt{time-in}[u] \leq \texttt{time-in}[v] \leq \texttt{time-out}[v] \leq \texttt{time-out}[u]$.*

Proof. If u is an ancestor of v, then v belongs to the subtree rooted at u. By **Lemma 3.1**, $t_v \subseteq [\texttt{time-in}[u], \texttt{time-out}[u])$. Indeed, $\texttt{time-in}[v], \texttt{time-out}[v] \in t_v$. □

Lemma 3.3 (Subtree size [10] [9])**.** *For every vertex u, the number of vertices in the subtree rooted at u, or S_u **Def. 2.8**, is $|V_u| =$ time-out$[u] -$ time-in$[u]$.*

Proof. The timer increases exactly once for each visited vertex. From **Lemma 3.1**, the subtree of u is represented by the interval [*time-in*$[u]$, *time-out*$[u]$], and by our definitions of *time-in*$[u]$ and *time-out*$[u]$, which tracks the moments when we first and last visit u. Therefore, *time-out*$[u]$ − *time-in*$[u]$ is exactly the number of vertices within the subtree rooted at u. □

4. Lowest Common Ancestor (LCA)

Now, let us move on to the next topic in this series, i.e. Lowest Common Ancestor, abbreviated as LCA. A handy algorithm to offer logarithmically effective answer to queries regarding two nodes in a tree T.

4.1. Theoretical foundation

Definition 4.1 (Lowest Common Ancestor [1] [11])**.** Let $T = (V, E)$ is a rooted tree, by **Def. 2.5**. For $w, u, v \in V$, w is the Lowest Common Ancestor of u and v, denoted as $LCA(u, v)$, if and only if u, v are descendants of the subtree rooted at w with the **smallest height** or the **biggest depth**.

There are a variety of solutions to approach this problem, each of which requires different time complexity. For a tree $T = (V, E)$ and $|V| = n$, there are, but not limited to:

1. **Sqrt–decomposition** [11]: $\mathcal{O}(n)$ preprocessing and $\mathcal{O}(\sqrt{n})$ per query.
2. **Binary Lifting (Sparse Table)** [9]: $\mathcal{O}(n \log n)$ preprocessing and $\mathcal{O}(\log n)$ per query.
3. **Euler Tour and Segment Tree** [11]: $\mathcal{O}(n)$ preprocessing and $\mathcal{O}(\log n)$ per query.
4. **Euler Tour and Sparse Table** [11]: $\mathcal{O}(n \log n)$ preprocessing and $\mathcal{O}(1)$ per query.

It is worthwhile to learn **Sqrt-decomposition, Binary Lifting, Segment Tree, and Sparse Table** in details, but these are not our primary goals. Hence, we will be discussing the the solutions that utilise **Euler Tour** and refer to **Sparse Table** and **Segment Tree** as abstract data structures. Still, we devote the following definitions below as references to our use of **Segment Tree** and **Sparse Table**.

Before delving into these two data structures, it is worth noting that they were originally made to solve a class of Range Query Problems. Interestingly, a LCA problem can be reduced to a Range Minimum Query (RMQ) problem.

Definition 4.2 (A RMQ problem [9] [11])**.** Given an array of n elements, denoted as $arr = [arr_0, \cdots, arr_{n-1}]$. Then, for $0 \leq l \leq r \leq n-1$, return the minimum of the sub–interval $[arr_l, \cdots, arr_r]$.

Lemma 4.1 (From LCA to RMQ [11] [9])**.** *LCA can be transformed into RMQ, and thereby, solvable by the utilisation of the same algorithms and data structures.*

Proof. Let us first flatten our rooted tree $T = (V, E)$ using the Euler Tour Technique introduced earlier. Let us name the arrays used in this proof, these arrays can be constructed during the **Euler Tour**:

- `euler[]`: stores the vertices visited by the `DFS()`.
- `first[]`: stores the first occurence of a vertex in the Euler Tour.
- `depth[]`: stores the depth of a vertex in T.

For $u, v \in V$ and $w = LCA(u, v)$. Without loss of generality, assume `first[u]` $\leq$ `first[v]`. By **Lemma 3.1**, $t_u, t_v \subseteq$ `[time-in[w],time-out[w]]`. Said differently, w is involved in the path from `first[u]` to `first[v]` on T. Therefore, the $LCA(u, v)$ now refers to the vertex w that has the lowest depth on $euler[first[u] \cdots first[v]]$. Formally, this is:

$$LCA(u, v) = euler[w], \qquad \text{where } w \doteq \underset{k \in euler[first[u], \cdots, first[v]]}{\arg\min} \{depth[k]\}$$

This becomes asking the minimum value on a range $[first[u] \cdots first[v]]$. □

Definition 4.3 (Segment Tree [11] [9])**.** For an array of n elements, denoted as $arr = [arr_0, \cdots, arr_{n-1}]$. Suppose having a query, namely $\mathcal{Q}(l, r) = f([arr_l, \cdots, arr_r])$, where $0 \leq l \leq r \leq n-1$. The array arr can be reconstructed as a Segment Tree subject to $f(\cdot)$. A Segment Tree is a complete binary tree [1]. It allows update and retrieval operations in $\mathcal{O}(\log n)$. The generic version of segment tree is presented below.

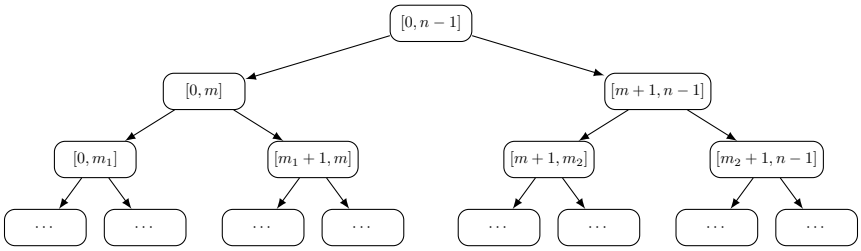

Figure 1: General structure of a segment tree.

A generic Segment Tree is organised as follows:

$$ST[l][r] = \begin{cases} arr[l], & l = r \\ f\left(ST\left[l\right]\left[\lfloor \frac{l+r}{2} \rfloor\right], ST\left[\lfloor \frac{l+r}{2} \rfloor + 1\right]\left[r\right]\right), & l < r \end{cases}$$

Segment Tree is a nice data structure that requires preprocessing in $\mathcal{O}(n)$ and allows intermediate updates. Simply put, we construct the Segment Tree from the `euler[]`, and at each node, it stores the minimum and the corresponding node of the original tree T. Consequently, it can answer in $\mathcal{O}(\log n)$.

Nonetheless, in almost LCA Problems, there are no updates required. Thus, we can achieve much better query speed, specifically in $\mathcal{O}(1)$ for most Range Query Problems, by the utilisation of Sparse Table.

Definition 4.4 (Sparse Table [11] [9])**.** For an array of n elements, denoted as $arr = [arr_0, \cdots, arr_{n-1}]$. Suppose having a query, namely $\mathcal{Q}(l, r) = f([arr_l, \cdots, arr_r])$, where $0 \leq l \leq r \leq n-1$. A Sparse Table, namely $st[][]$, can be constructed from arr by the utilisation of the recursion relation as follows:

$$st[j][i] = \begin{cases} arr[i] & j = 0 \\ st[j-1][i] \ \cup \ st[j-1][i + 2^{j-1}] & j > 0 \end{cases}$$

Therefore, $st[j][i] = f([i \cdots i + 2^j - 1])$. In other words, $st[j][i]$ answers the query of an interval starting at i^{th} with length of $(i + 2^j - 1) - i + 1 = 2^j$. Consequently, a query $\mathcal{Q}(l, r)$ is returned in $\mathcal{O}(1)$ by:

$$\mathcal{Q}(l, r) = f\left(\left[l \cdots l + (2^k - 1)\right]\right) \bigcup f\left(\left[r - 2^k + 1 \cdots (r - 2^k + 1) + (2^k - 1)\right]\right)$$

where $k \doteq \lfloor \log_2(r - l + 1) \rfloor$.

More on Sparse Table From the definition above, one may realise that, for a query $\mathcal{Q}(l, r)$, the data structure does not store the answer for $\log_2(r - l + 1) \notin \mathbb{N}$. In other words, if the length of the interval starting at l is not a power of 2, the answer is not explicitly retained in $st[][]$. Therefore, we resolve this issue by choosing $k = \lfloor \log_2(r - l + 1) \rfloor$, such that:

$$f\left([l, r]\right) = f\left(\left[l \cdots l + 2^k - 1\right]\right) \bigcup f\left(\left[r - 2^k + 1 \cdots r\right]\right)$$

Possibly, $r - 2^k + 1 \leq l + 2^k - 1$. Hence, this only works in $\mathcal{O}(1)$ for **idempotent operators**, e.g. $\min$, $\max$, lcm, gcd, and others. In contrast, for non–idempotent operators, e.g. $\sum$ and $\prod$, this can only be resolved in $\mathcal{O}(\log n)$ [9] [11].

4.2. Algorithmic design

One can approach this problem straightforwardly as follows. We perform a DFS from the root to determine the height and the parent node for every vertex in the tree. Then, for each query, we ascend either u or v towards the root until we achieve two new nodes u and v with the same height. From there, we can simultaneously ascend two vertices towards the root until they reach one same vertex, called w, which turns out to be the answer to our query. Nevertheless, this would cost one query completed in $\mathcal{O}(n)$, as a tree can be a simple path. Therefore, an inefficient solution when the number of queries are large.

Thanks to Lemma 4.1, we can convert our primal LCA problems into RMQ problems. Therefore, we can efficiently solve this problem using **Euler Tour algorithm**, and subsequently applying either Segment Tree or Sparse Table.

For the LCA problem, the Euler Tour must also record the vertex reached after each recursive return. Thus, a vertex may appear several times in the array. We then store the first position of each vertex in the tour and build a Sparse Table over the depth values of the tour. Since the RMQ operator is $\min$, which is idempotent, each LCA query can be answered in $\mathcal{O}(1)$ after preprocessing.

A high–level algorithmic design of the solution is presented as below.

Algorithm 2 LCA preprocessing by Euler Tour and Sparse Table [11] [9]

```
function BUILDLCA(root)
    EULERTOUR(root)                                   ▷ Obtain euler[], first[], depth[]
    SEGMENTTREE(euler[], first[], depth[])            ▷ Obtain ST[l][r]
    SPARSETABLE(euler[], first[], depth[])            ▷ Obtain st[j][i]
end function
```

Algorithm 3 Answering an LCA query

```
function LCA(u, v)
    l ← first[u]
    r ← first[v]
    if l > r then
        swap l and r
    end if
    k ← ⌈log(r − l + 1)⌉
    answer ← st[k][l] ∪ st[k][r − 2^k + 1] OR answer ← SEGMENTTREEQUERY_{ST[][]}(l, r)
    return answer.node
end function
```

Complexity Analysis EULERTOUR() is done in $\mathcal{O}(m+n)$, where $m = |E|$ and $n = |V|$. Then, depending on whichever the data structure is used:

- Segment Tree: preprocessing $\mathcal{O}(n)$ and query $\mathcal{O}(\log n)$.
- Sparse Table: preprocessing $\mathcal{O}(n \log n)$ and query $\mathcal{O}(1)$. As no updates are required.

5. Heavy–Light Decomposition (HLD)

Now, let us move on to another fascinating technique to decompose a tree for more efficient queries on it. Previously, we introduce how to flatten a tree by Euler Tour technique, and leveraging LCA to resolve problems regarding two nodes, i.e. the path connecting two vertices in a tree T. Nevertheless, when it comes to queries on that path, LCA is critically inefficient, as its complexity is linearly proportional to the length of the path.

5.1. Theoretical foundation

As the name implies, in this technique, we are working with heavy and light notions. One may familiarise with heavy subtrees in AVL trees [1]. This is quite relevant, except that there are few novel notions about heavy and light edges.

Definition 5.1 (Heavy and light child [10])**.** Consider a rooted tree $T = (V, E)$. For a vertex $v \in V$ and its children u, a heavy child u_j is the one with the most nodes residing in the subtree rooted at u_j, whereas other children u_i are called light children.

Definition 5.2 (Heavy and light edges [10] [9])**.** Consider a rooted tree $T = (V, E)$. For a vertex $v \in V$ and its children u. A heavy edge connects v and u_j, the heavy child, whereas light edge connects v and other light children u_i.

Noticeably, there are a slightly different notion of heavy and light edges, as specified in [11]. In particular, they define a heavy edge as an edge that connects to a child with a subtree size rooted at u_i that is greater than or equal to the half of size of the subtree rooted at v, formally $|V_{u_i}| \leq \frac{|V_v|}{2}$. Nonetheless, for ease of implementation, we refer to heavy edges are those connecting v and the child with the largest subtree size rooted at that child and randomly designated to children with equal subtree size. Indeed, later on, [11] also mentioned this in their article.

Definition 5.3 (Heavy paths [10] [9] [11])**.** A heavy path is a maximal and consecutive set of heavy edges. Indeed, all nodes of the tree are involved in a heavy path. There are no so–called light paths, since these are deemed as light edges linking the heavy paths.

From the definitions of heavy paths and light edges, we can perform path queries between any arbitrary vertices by processing each light edge on the path and processing heavy paths in logarithmic time complexity with the utilisation of Segment Tree. Consequently, this yield an overall complexity as $\mathcal{O}(\log^2 n)$, where $|V| = n$.

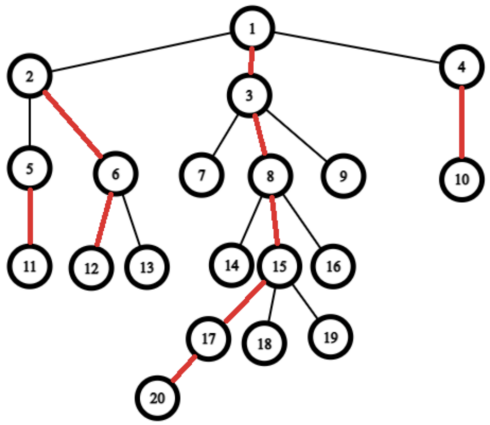

Figure 2: An example of HLD on a tree rooted at 1. Red and black indicate heavy and light edges, respectively. [9]

Theorem 5.1 (Upperbound of light edges [10] [9] [11])**.** *For a tree $T = (V, E)$ and two vertices $u, v \in V$, the path connecting u and v consists at most $\mathcal{O}(\log n)$ light edges.*

Proof. Suppose there is a tree $T = (V, E)$ and two vertices $u, v \in V$. We can easily find the path from u to v in $\mathcal{O}(\log n)$ by the utilisation of **LCA**, denoted as $w = LCA(u, v)$. Hence, we perform similar steps on either the path from $u \leftrightarrow w$ and $v \leftrightarrow w$. Without loss of generality and for the sake of convenience, consider the path from $u \leftrightarrow w$.

Let k_u be the direct child of w involved on the path from $w \rightarrow u$. Suppose $e_{w \rightarrow k_u} \in E$ is a light edge. By **Def. 5.2**, the number of vertices in S_{k_u} is reduced at least half of the size of S_w, i.e.

$$|V_{k_u}| \leq \frac{1}{2}|V_w|$$

Recursively, we consider the direct child of k_u, supposing the edges connecting them are light and descend until we reach u. Since $|V_w| \leq |V|$, where $|V| = n$, and we terminate this process as long as it reaches u, or in the worst scenario, u is the leaf of T, i.e. $|V_u| = 1$. Therefore, the number of steps is at most $\mathcal{O}(\log n)$ light edges. Now, the sum of such light edges on both path from $w \leftrightarrow u$ and $w \leftrightarrow v$ are $\mathcal{O}(\log n) + \mathcal{O}(\log n) = \mathcal{O}(\log n)$. □

Corollary 5.1 (Upperbound of heavy paths [10] [9] [11])**.** For a tree $T = (V, E)$ and two vertices $u, v \in V$, the path connecting u and v consists at most $\mathcal{O}(\log n)$ heavy paths.

Proof. This is an immediate result from **Theorem 5.1**. According to our notion of light edges and heavy paths from **Def. 5.2** and **Def. 5.3**, the heavy paths are connected via a light edge, otherwise, we can simply jump in the heavy path. Since there are at most $\mathcal{O}(\log n)$ light edges, and thus, the number of heavy paths are bounded by $\mathcal{O}(\log n)$. □

5.2. Algorithmic design

In this section, we deliver fairly high-level narrative and implementation of the HLD algorithm and how it produces answer to queries. First and forthmost, we leverage DFS() to obtain the parent of vertices in rooted tree $T = (V, E)$ and size of (sub)trees. Next, we decompose the tree into heavy and light edges by the utilisation of HLD(). Eventually, we construct a SEGMENTTREE() to support fast operation on nodes belong to heavy paths, while manually processing light edges.

One may come with two approaches for heavy paths. On one hand, we can independently construct a **Segment Tree** for each path, and thus still gives $\mathcal{O}(\log k)$, where k is the number of nodes involved in the path. However, there is a nice trick to construct a single **Segment Tree** in lieu of many independent ones. To do this, we simply convert nodes in T to an array, where nodes within the same heavy paths are consecutively arranged in the array, called `tree2array`. Then, for sake of converting back to original nodes, we maintain the `arr2tree[]`. Now, we can introduce `chainHead[]` and `chainId[]` to track the leading index of a heavy path and which heavy path the current node belongs to, respectively. As a result, we build the **Segment Tree** upon the `tree2arr[]` and convert back and forth with the use of `arr2tree[]`.

Regarding HLD(), it is quite similar to a usual DFS(), as we recursively descend to heavy edges until there are no heavy edges left. From there, we move to the next heavy path. An detailed implementation of `HLD()` is presented below, followed by high-level preprocessing and query answering.

Algorithm 4 HLD [9]

```
curChain ← 1
curPos ← 1
function HLD(u, p)
    if chainHead[u] = ∅ then
        chainHead[u] ← u
    end if
    chainId[u] ← curChain
    tree2arr[u] ← curPos
    arr2tree[curPos] ← u
    curPos ← curPos + 1
    nxt ← 0
    for all v ∈ N(u) do
        if v ≠ p then
            if nxt = 0 or size[v] > size[nxt] then
                nxt ← v
            end if
        end if
    end for
    if nxt ≠ 0 then
        Hld(nxt, s)
    end if
    for all v ∈ N(u) do
        if v ≠ p and v ≠ nxt then
            CurChain ← CurChain + 1
            Hld(v, s)
        end if
    end for
end function
```

Algorithm 5 Answering a path query by HLD [9]

```
function PathQuery(u, v, is_update)
    if is_update = True then
        UpdateVertex(u, v)
    else
        w ← LCA(u, v)
        answer ← ∅                                  ▷ Assume no valid answer
        answer_u ← SubPathQuery(u, w)
        answer_v ← SubPathQuery(v, w)
        answer ← answer_u ∪ answer_v
        return answer
    end if
end function
```

```
function SubPathQuery(u, w)
    answer ← ∅                                      ▷ Assume no valid answer
    while chainId[u] ≠ chainId[w] do
        l ← tree2arr[chainHead[chainId[u]]]
        r ← tree2arr[u]
        answer ← answer ∪ SegmentTreeQuery_ST[][](l, r)
        u ← par[chainHead[chainId[u]]]
    end while
    return answer
end function
function UpdateVertex(u, value)
    Update the ST[][] at tree2arr[u] to value
end function
```

Algorithm 6 HLD preprocessing [9]

```
function BuildHLD(T, root)
    Root the tree T at root
    DFS(T, ROOT)                                    ▷ Obtain par[], size[]
    HLD(root, −1)
                                                    ▷ Obtain chainHead[], chainId[], tree2arr[], arr2tree[] from HLD()
    SegmentTree(chainHead[], chainId[], tree2arr[], arr2tree[])
                                                    ▷ Obtain ST[][] from SegmentTree()
end function
```

Complexity Analysis Let us break down two stages of the HLD algorithm.

- Preprocessing: The first DFS computes subtree sizes and heavy children in $\mathcal{O}(m+n)$, where $n = |V|$ and $m = |E|$. The decomposition traversal also takes $\mathcal{O}(n)$, and the Segment Tree is built in $\mathcal{O}(n)$. Therefore, the preprocessing complexity is $\mathcal{O}(n)$.
- Query: According to **Corollary 5.1**, every path contains at most $\mathcal{O}(\log n)$ heavy paths. Each Segment Tree query over one heavy path costs $\mathcal{O}(\log n)$; hence, one path query costs $\mathcal{O}(\log^2 n)$. A single vertex update costs $\mathcal{O}(\log n)$.

Note that, for a single Segment Tree, the query is done in $\mathcal{O}(\log n)$, not $\mathcal{O}(\log k)$ as in many independent Segment Trees.

6. Centroid Decomposition

Finally, we move to another advanced tree decomposition algorithm, called Centroid Decomposition, which is primarily inspired by divide–and–conquer paradigm [11]. This technique is pivotal important to problems relating, but not limited to counting paths and finding distances subject to some given properties. Unlike HLD, which does not reconstruct the tree, this algorithm directly alters the original structure of the tree, and thus, faciliate computations for specific tasks, which HLD is neither efficient nor feasible.

A typically classic problem is, given a tree $T = (V, E)$, counts all paths with length exactly k [11] [9]. This problem statement serves as the very first impression of the capability of Centroid Decomposition.

6.1. Theoretical foundation

Before proceeding to further, let us introduce relevant definitions and theorems. In addition, we provide the following figure as an intuition of Centroid Decomposition.

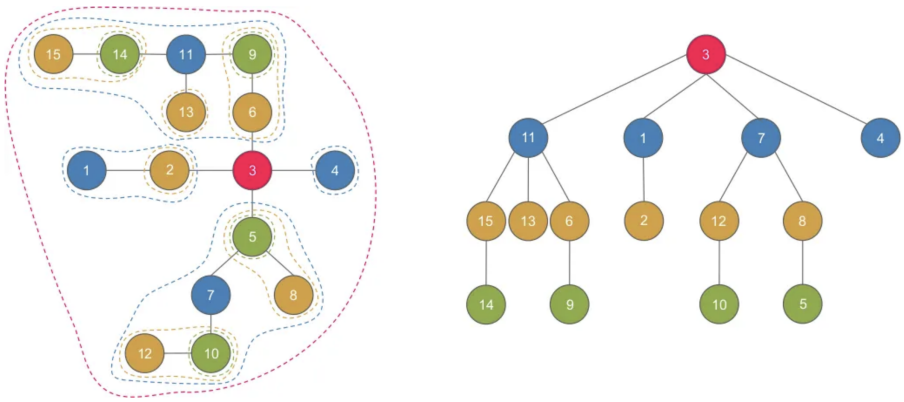

Figure 3: An example of centroid decomposition. Left figure is the original tree, right figure is the centroid tree [12].

Definition 6.1 (A centroid of tree [9] [10] [11])**.** For a tree $T = (V, E)$, a vertex $c \in V$, then c is deemed as a centroid if removing it results in subtrees having fewer than half of the nodes in the original tree.

The new tree obtained from decomposing the original tree is called a **centroid tree** [11] [9]. Noticeably, **center** and **centroid** of a tree are two completely different notions.

Definition 6.2 (A center of tree [12])**.** For a tree $T = (V, E)$, a vertex $c \in V$ is called a center of T is it minimizes the largest distance from other nodes to itself. Formally, this is:

$$c = \arg\min_{u \in V} \left\{ \max_{v \in V} d(u, v) \right\}$$

where $d(u, v)$ is the number of vertices involved from u to v in T, including themselves. Indeed, there are possibly more than one centers.

Theorem 6.1 (Existence of a centroid [11])**.** *For a tree $T = (V, E)$, there is always either one or two centroids.*

Proof. For a tree $T = (V, E)$, not necessarily rooted. Given an algorithm starting at any arbitrary vertex $v \in V$, it continuously moves to the heavy child of v **Def. 5.1** until no child subtree has more than $\frac{|V|}{2}$ nodes. If this algorithm always terminates, the proof is completed.

At a current point v, the algorithm can either, moves to $u \in N(v)$ if $s(S_u) > \frac{s(S_v)}{2}$ vertices, or terminates if there are none. Trivially, if it terminates, the proof is completed. On the other hand, every time it descends to a heavy child, the size of the subtree is reduced by an arbitrary number of nodes. Of course, it cannot return to its parent, otherwise, it terminates. Consequently, this algorithm always terminates, as the number of nodes is monotonically decreased at each step. Thus, we complete the proof. □

Corollary 6.1 (Two centroids [11])**.** For a tree $T = (V, E)$, if there are two centroids, they are always directly connected.

Proof. This can be proven by contradiction. For a tree $T = (V, E)$, suppose there are two centroids $c_1, c_2 \in V$, such that they are not adjacent. Without loss of generality, let us remove c_1, and thus, we are left with subtrees with at most $\frac{|V|}{2}$ nodes. Now, consider the subtree that c_2 is lying therein. Because they are not directly connected, there are some nodes $u \in V$ on the path from c_1 to c_2. By **Def. 6.1**, c_2 is the vertex where the current tree can be partitioned such that no resulting subtrees having more than $\frac{|V|}{2}$ nodes. Therefore, we are left with a subtree,

where c_2 is in, having more than $\frac{|V|}{2}$ nodes, which violates the condition guaranteed by centroid c_1. Thus, they must be adjacent. Moreover, two centroids can only exist when $|V|$ is an even number. □

Theorem 6.2 (Decomposition depth [11])**.** *For a tree $T = (V, E)$, its corresponding centroid tree $T' = (V, E')$ has at most $\mathcal{O}(\log n)$ levels, where $n = |V|$.*

Proof. Consider a tree $T = (V, E)$, for a vertex $v \in V$, the number of levels in the centroid tree is the maximal number of decomposition steps until v is chosen as the centroid. Through each decomposition step i^{th}, v belongs to a subtree having at most $\frac{|V|}{2^i}$, explitcitly, this is:

$$
\begin{aligned}
&\text{At } 0^{\text{th}} \text{ step: } v \in T = (V, E) \text{ where } |V| = n \\
&\text{At } 1^{\text{st}} \text{ step: } v \in T' = (V', E') \text{ where } |V'| < \frac{|V|}{2} \\
&\text{At } 2^{\text{nd}} \text{ step: } v \in T'' = (V'', E'') \text{ where } |V''| < \frac{|V'|}{2} < \frac{|V|}{2^2} \\
&\qquad\qquad \vdots \\
&\text{At } k^{\text{th}} \text{ step: } v \in T^{(k)} = (V^{(k)}, E^{(k)}) \text{ where } |V^{(k)}| < \frac{|V^{(k-1)}|}{2} < \cdots < \frac{|V|}{2^k}
\end{aligned}
$$

Consequently, the decomposition terminates, at most, when $|V^{(k)}| = 1$ or $2^k = n \Leftrightarrow k = \log n$. □

Theorem 6.3 (Path coverage [11] [12])**.** *For a tree $T = (V, E)$, let $u, v \in V$. Consider its corresponding centroid tree $T' = (V, E')$, let $w = LCA_{T'}(u, v)$ be the LCA of u and v in the centroid tree T'. Then, the path from u to v in the original tree T must pass through $w \in T'$.*

Before proceeding to the proof, it is noteworthy that, this is not a trivial result. Indeed, we are saying about the path P from u to v in the original tree T, where as w is the LCA of u and v in the centroid tree T'. A nice induction-based proof is introduced in [11]. Nevertheless, we will approach the proof from another aspect.

Proof. Suppose the centroid c_i at decomposition step i^{th} is not involved in P. Then, P must completely belong to a subtree, otherwise, it must pass through c_i. However, by **Theorem 6.2**, the subtree consisting P is monotonically reduced at every decomposition steps, until the subtree size is reduced to 1. Thus, at some step j^{th}, P contains the remaining nodes in a subtree. By **Theorem 6.1**, this subtree P always has at least a centroid, and thus, it must be decomposed into two disjoint parts, where $c_j = w = LCA(u, v)$. If not, either u or v is the $LCA(u, v)$. Otherwise, it violates the **Def. 4.1 Lowest Common Ancestor**. □

6.2. Algorithmic design

In this section, we present a generic high-level template of Centroid Decomposition-related problems. The core idea is to repeatedly locate a centroid of the current component, process information related to that centroid, remove it from the original tree, and recursively apply the same process to every remaining component. During this process, we maintain `removed[]` to mark vertices already chosen as centroids, `subtreeSize[]` to compute component sizes, and `centroidParent[]` to store edges of the resulting centroid tree. The exact content of PROCESSCENTROID() depends on the problem being solved.

Algorithm 7 Generic Centroid Decomposition template [11] [9]

```
function CentroidDecomposition(root)
    Initialize removed[] with False
    Initialize centroidParent[] with ∅
    return Decompose(root, −1)
end function
function Decompose(entry, parent)
    componentSize ← ComputeSubtreeSize(entry, −1)
    c ← FindCentroid(entry, −1, componentSize)
    centroidParent[c] ← parent
    ProcessCentroid(c)                          ▷ Problem-specific operation
    removed[c] ← True
    for all v ∈ N(c) do
        if removed[v] = False then
            Decompose(v, c)
        end if
    end for
    return c
end function
```

```
function ComputeSubtreeSize(u, parent)
    subtreeSize[u] ← 1
    for all v ∈ N(u) do
        if v ≠ parent and removed[v] = False then
            subtreeSize[u] ← subtreeSize[u] + ComputeSubtreeSize(v, u)
        end if
    end for
    return subtreeSize[u]
end function
function FindCentroid(u, parent, componentSize)
    for all v ∈ N(u) do
        if v ≠ parent and removed[v] = False then
            if subtreeSize[v] > componentSize/2 then
                return FindCentroid(v, u, componentSize)
            end if
        end if
    end for
    return u
end function
function ProcessCentroid(c)
    Collect or update information involving centroid c
    Combine information from components adjacent to c
    Store the result required by the target problem
end function
```

In Algorithm 7, Decompose() is the main recursive routine. First, it computes the size of the current component, then locates a centroid according to **Def. 6.1**. After the centroid is selected, ProcessCentroid() is called as a placeholder for the actual problem-dependent logic. Finally, the centroid is marked as removed, and the algorithm is recursively applied to each remaining component.

Complexity Analysis For each centroid level, every vertex in the current component is visited a constant number of times by subtree computation and centroid search. By **Theorem 6.2**, the centroid tree has at most $\mathcal{O}(\log n)$ levels. Therefore, the generic decomposition template requires $\mathcal{O}(n \log n)$ time. The total complexity may increase subject to the implementation of ProcessCentroid().

7. Conclusion

Throughout this article, we have reviewed several important techniques and algorithms regarding a special class of graph, i.e. Trees. Starting from fundamental graph-theoretic notions, we moved towards Euler Tour, LCA, HLD, and Centroid Decomposition, each of which exploits a different structural property of trees. Euler Tour converts subtree relations into interval relations, LCA identifies the meeting point of two vertices in a rooted tree, HLD breaks paths into logarithmically many heavy segments, and Centroid Decomposition recursively separates a tree into balanced components.

These are fairly advanced techniques and algorithms of trees. Still, this is an open–ended domain, where many more advancements and ideas can flourish. Some of other fascinating tree-related data structures are, but not limited to, [9] Fenwick Tree, Interval Tree, Range Tree, Li–Chao Tree, [10] Link-Cut Tree, Kruskal Reconstruction Tree, and Virtual Tree.

Although the techniques introduced in this article are closely related, they are not interchangeable. The following table summarizes their main distinctions.

Table 1: Comparison between LCA, HLD, and Centroid Decomposition

Technique	**Main purpose**	**Core idea**	**Typical use**
LCA	Locate the meeting ancestor of two vertices.	Reduce ancestry to RMQ or binary lifting.	Distances, path identity, and tree subroutines.
HLD	Answer path queries and updates.	Split paths into logarithmically many heavy segments.	Vertex or edge values along paths.
Centroid Decomposition	Solve global problems recursively.	Remove centroids to form balanced components.	Distance counting and divide-and-conquer tree problems.

References

[1] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*. Wiley, 6 ed., 2014.

[2] L. W. Beineke and R. J. Wilson, eds., *The History of Graph Theory: A Century of Progress*. Oxford, UK: Oxford University Press, 2001.

[3] K. Appel and W. Haken, "Every planar map is four colorable," *Illinois Journal of Mathematics*, vol. 21, no. 3, pp. 429–567, 1977.

[4] D. A. Marcus, *Graph Theory: A Problem Oriented Approach*. Washington, DC, USA: Mathematical Association of America, 2008.

[5] X. Zhang, S. Li, H. Li, P. Gong, J. Shi, J. Shen, C. Tian, D. Zhang, and Q. Zhu, "A survey on graph neural networks," *IEEE Access*, vol. 11, 2023.

[6] R. Diestel, *Graph Theory*. Berlin, Germany: Springer, 5 ed., 2017.

[7] K. R. Saoub, *Graph Theory: An Introduction to Proofs, Algorithms, and Applications*. Boca Raton, FL, USA: Chapman and Hall/CRC, 2021.

[8] W. Goddard, "A brief introduction to discrete mathematics." https://people.computing.clemson.edu/~goddard/texts/discreteMath/fullset2024.pdf, 2023. Accessed: 2026-05-03.

[9] VNOI Wiki, "Vnoi wiki." https://wiki.vnoi.info, 2024. Accessed: 2026-05-05.

[10] USACO Guide, "Usaco guide." https://usaco.guide, 2024. Accessed: 2026-05-05.

[11] CP-Algorithms, "Cp-algorithms." https://cp-algorithms.com, 2024. Accessed: 2026-05-05.

[12] I. Carpanese, "An illustrated introduction to centroid decomposition." https://medium.com/carpanese/an-illustrated-introduction-to-centroid-decomposition-8c1989d53308, 2018. Medium article, accessed May 9, 2026.

[13] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. MIT Press, 2009.

[14] D. Grinberg, "An introduction to graph theory," 2025.